

- Write a C program to Create a Binary Tree or Binary search tree and display.
- Write a C program to sort n elements using Binary tree Sort technique.
- Write a C program to implement three TREE traversal methods.

```
#include<stdio.h>
struct node
{
    int info;
    struct node* left;
    struct node* right;
};
typedef struct node* nodeptr;
nodeptr getnode()
{
    return (nodeptr)malloc(sizeof(struct node));
}
nodeptr maketree(int x)
{
    nodeptr p;
    p=getnode();
    p->info=x;
    p->left=NULL;
    p->right=NULL;
    return (p);
}
void main()
{
    nodeptr root,p,q;
    int number;
    char ch;
    printf("Enter the root:");
    scanf("%d",&number);
    root = maketree(number);
    do
    {
        p=q=root;
        printf("Enter a number:");
        scanf("%d",&number);
        while(number!=p->info && q!=NULL)
        {
            p=q;
            if(number<=p->info)
                q=p->left;
            else
                q=p->right;
        }
        if(number == p->info)
            printf("\n%d is a duplicate value",number);
        else if(number<p->info)
            setleft(p,number);
        else
            setright(p,number);
        printf("\nDo you want to continue(y/n)?");
        scanf("%s",&ch);
    } while(ch=='y');
    printf("\nInorder:");
    inorder(root);
}
```

```

    printf("\nPreorder:");
    preorder(root);
    printf("\nPostorder:");
    postorder(root);
}

void setleft(nodeptr p,int x)
{
    if(p==NULL)
        printf("Cant insert");
    else if(p->left != NULL)
        printf("Invalid insertion");
    else
        p->left = maketree(x);
}

void setright(nodeptr p,int x)
{
    if(p==NULL)
        printf("Cant insert");
    else if(p->right != NULL)
        printf("Invalid insertion");
    else
        p->right = maketree(x);
}

void inorder(nodeptr p)
{
    if(p!=NULL)
    {
        inorder(p->left);
        printf("%d\t",p->info);
        inorder(p->right);
    }
}

void preorder(nodeptr p)
{
    if(p!=NULL)
    {
        printf("%d\t",p->info);
        inorder(p->left);
        inorder(p->right);
    }
}

void postorder(nodeptr p)
{
    if(p!=NULL)
    {
        inorder(p->left);
        inorder(p->right);
        printf("%d\t",p->info);
    }
}

```